

VibeGuardBench: A Deterministic Benchmark for Repair-and-Regression in AI-Generated Web Applications

Cipher Dev (Independent Researcher) - October 2026

Abstract

Prompt-driven AI application builders can produce convincing full-stack web applications from a single specification. Existing benchmarks score that first act: what the builder generates. What happens next -- when deterministic engineering feedback identifies concrete defects and the builder is asked to fix them -- is unmeasured. VibeGuardBench is an open benchmark for that second act. Three frozen application specifications (appointment booking, multi-role dashboard, ordering/inventory) are generated by commercial AI builders; a deterministic 39-check oracle suite (build, types, lint, functional, authorization, security, accessibility, persistence) measures each application; machine-generated feedback is delivered per failing check; and the suite re-executes after every repair, for up to three iterations. The benchmark reports repair success rate, regression rate among previously passing checks, and net quality change, with raw counts and a preregistered analysis plan. A harness-validation pilot (one application, excluded from research claims) exercised the full loop and surfaced fifteen harness defects before data collection. The benchmark, oracles, schemas, and analysis scripts are open source.

Keywords: IEEEkeywords

AI-generated code, vibe coding, benchmarks, automated program repair, regression testing, empirical software engineering

Keywords: IEEEkeywords

Introduction

The evaluation of AI programming tools has a blind spot, and it sits exactly where practitioners live. Benchmarks such as WebGen-Bench~[2], FullStackBench~[3], and AppWorld~[4] score what an AI builder produces from a specification. SWE-bench and its successors~[5] score patches against unit tests in human repositories. None of them measures the ordinary, expensive middle of real software work: the generated application fails objective checks, someone hands the failures back, and the builder tries again. Does the fix work? And what else broke in the process? Classical automated program repair learned this lesson the hard way. Patches that satisfy a test oracle frequently break behavior elsewhere -- documented as test overfitting~[9], patch overfitting~[11], and preservation violations~[10]. Whether modern AI application builders exhibit the same behavior when repairing their own output, at the level of a whole

application with functional, security, accessibility, build, and persistence concerns, has not been measured. VibeGuardBench exists to measure it. The benchmark's research question is deliberately narrow: *when deterministic engineering feedback identifies failures in an AI-generated full-stack application, can the same AI builder repair those failures without introducing regressions in previously passing functionality?* Everything in the design serves that question. Three commitments define it. First, : every check produces the same outcome given the same artifact; no LLM judges anything. Second, the : after every repair, all checks re-execute, because regressions are only visible to checks that run. Third, : specifications, prompts, oracles, metric definitions, and the analysis plan were committed publicly before any experimental data existed. This paper describes the benchmark as a tool: the tasks, the oracle suite, the repair protocol, the metrics, the implementation, and the pilot that validated the harness. A companion protocol document~[19] records the frozen study design; this paper is about the instrument itself.

Related Work

Benchmarks for AI-generated software cluster around initial generation. WebGen-Bench~[2] scores generated websites with automated tests; FullStackBench~[3] spans full-stack task domains; AppWorld~[4] verifies agent programs against simulated applications; DevBench~[] covers the development lifecycle. SWE-bench~[5] and its agents~[6,7,8] grade patches to human repositories against fail-to-pass tests -- close in spirit to repair, but the subject is a patch to existing code, not a builder repairing its own generated application, and the oracle is a unit-test subset rather than a whole-application suite. The repair literature proper is older and sobering. Smith et al.~[9] showed that most "repaired" programs in a large APR study were not actually correct; subsequent work formalized overfitting as the central pathology of test-driven repair~[11,12,10]. Self-repair loops for LLM code (Self-Refine~[13], Self-Correct~[14], Reflexion~[15]) report improved pass rates but rarely re-run a full behavioral suite, so regressions are structurally undercounted -- the exact flaw the whole-suite rule is designed to prevent. On security evaluation, Pearce et al.~[16,17] and Perry et al.~[18] established that LLM-generated code carries distinctive vulnerability patterns; VibeGuardBench folds a frozen static ruleset and probe-based checks into the suite rather than treating security as a separate study. A recent survey of vibe coding~[1] identifies reliability and security of generated products as open empirical problems; this benchmark is a direct instrument for both.

Benchmark Design

The design is a factorial grid with a longitudinal component at the check level: 3 application tasks 2 commercial builders 3 independent repetitions, yielding 18 initial applications (iteration 0), each eligible for up to 3 repair iterations. The statistical unit is the check observation (application iteration check): 39 checks in the catalog, roughly 20-25 eligible per suite execution, on the order of 1,500-2,000 observations across 72 executions.

Application tasks..

repository: APP-01, an appointment booking system (two roles, CRUD, search, admin panel); APP-02, a multi-role SaaS dashboard (three roles, project-task hierarchy, role-scoped API); APP-03, an ordering/inventory application (order placement with stock decrement, manager-only inventory administration). Each specification carries a binding -- exact HTTP routes, request/response JSON

shapes, seed accounts, DOM `data-testid` attributes -- which is what makes deterministic cross-implementation testing possible. Builders keep freedom over stack, architecture, and visual design; contract deviations fail the affected checks, and that failure is a measured outcome, not a setup error.

Repair loop..

one fixed-template message per failing check -- deterministic text with observed versus expected values and a specification reference -- preceded by a preamble instructing repair without changing unrelated behavior. The operator delivers the feedback verbatim to the same builder session, exports the result, and the runner re-executes the entire suite. Iteration 0 is archived once and never overwritten; repairs stop early only when nothing is failing.

The Oracle Suite

Thirty-nine checks across eight categories (Table~), each with a frozen identifier from the check catalog. The runner installs, builds, launches, and probes the application automatically, recording a schema-validated run record with raw tool output per check. [tbp] when the application cannot start (cascade rule). 4pt llX Category & \# & Method \ BUILD & 5 & fresh install; production build; start on PORT; reachability; log hygiene \ TYPE & 1 & `tsc -noEmit` \ LINT & 2 & ESLint, frozen config; zero errors / zero warnings \ FUNC & 11 & Playwright (Chromium): login, logout, create, list, edit, delete, validation, search, empty state, not-found, API rejection \ AUTHZ & 6 & role API probes; cross-owner writes; UI denial; logout invalidation \ SEC & 6 & frozen Semgrep ruleset; secret scan; `npm audit`; debug probes; cookie/token storage; credential storage \ A11Y & 4 & axe-core (WCAG 2.1 A/AA subset); lang/title \ DB & 4 & create/update/delete survive process restart; contract schema shapes \ Three interpretation rules matter. unstartable applications record runtime checks as not applicable; the install/build failures carry the signal. environment-class failures are retried once and logged; test-content failures never are. scanner findings are candidate findings, never reported as confirmed vulnerabilities; every distinct finding is manually inspected, with a false-positive sensitivity analysis in the frozen plan. No CVSS scores, no exploitability claims from scanner output alone.

Metrics

Per application a and transition $k \rightarrow k+1$, let $T(a,k)$ be failing checks and $P(a,k)$ passing checks at iteration k (environment-error and not-applicable checks excluded): $|T(a,k)| = (c) = ||T(a,k)||$, $|P(a,k)| = (c) = ||P(a,k)||$, $|T(a,k)| + |P(a,k)| = ||T(a,k) \cup P(a,k)||$. RSR (repair success rate), RR (regression rate), and NQC (net quality change) are the primary estimands. Raw counts accompany every rate; small denominators are flagged; application-level clustering handles the non-independence of checks within an application. The analysis plan is estimation-first with cluster bootstraps, pre-declared hypothesis families with Benjamini-Hochberg correction, and effect sizes as risk differences -- frozen before data collection.

Implementation

The repository ships the complete instrument: frozen specifications and prompts; the 39-check catalog with per-check feedback templates; oracle implementations (Playwright suites, a runtime probe engine, the Semgrep ruleset, ESLint config); `run\_oracles.py` (full-suite executor), `make\_feedback.py`

(deterministic feedback), ``ledger.py`` (raw processed); ``metrics.py`` and ``plots.py`` (analysis); JSON schemas for run records and check results; and an operator runbook for the builder-facing steps. Every suite execution writes environment versions, raw tool outputs, operator interventions, and retries into the run record. Every number in the results regenerates from raw data by script; none are transcribed by hand.

Pilot Validation

Before any study data, the harness was validated end to end on a pilot application: APP-01 generated by a non-study LLM system (explicitly excluded from research claims), three full suite executions, two real repair loops. The corrected trajectory: 19 passing / 16 failing at iteration 0; a repair that changed code yet fixed 0 of 16 failures while regressing 1 of 19 passing checks (the builder moved the session token into a cookie without the required ``HttpOnly`/`SameSite`` flags); and a correct repair of that regression at iteration 2. The pilot's value was not the trajectory but what it exposed: fifteen harness defects -- a lint configuration producing spurious errors, an over-strict health check misreading redirects, orphaned server processes, an invalid scanner configuration, silent pass-through of dependency-audit failures, cross-iteration state pollution, and nine more -- each fixed and logged before data collection. A benchmark that has not survived contact with a real repair loop is a specification, not an instrument; the pilot is what makes VibeGuardBench the latter.

Limitations

Two builders, three tasks, one operator bound generalization; conclusions concern the studied systems under the studied contracts. The prescriptive interface contract trades ecological representativeness for measurement validity. Builders are compared as systems, not models, since internals are undisclosed. Three repair iterations bound trajectory claims. Platform versions will drift during data collection; run records are the only reliable account of which version produced which artifact.

Conclusion

Initial-generation benchmarks measure the first act of AI-assisted development. VibeGuardBench instruments the second: repair under deterministic feedback, with regressions counted by construction rather than by accident. The benchmark is open, the oracles are frozen, the analysis is preregistered, and the harness has survived a real repair loop. The study it enables -- whether AI application builders can fix their own output without breaking what already works -- is, to our knowledge, unmeasured anywhere else.

References

- [1] Y.~Ge et al., "A survey of vibe coding with large language models," *arXiv:2510.12399*, 2025.
- [2] J.~Lu et al., "WebGen-Bench: Evaluating LLM-generated web applications," 2025.
- [3] ByteDance, "FullStackBench," 2025.
- [4] H.~Trivedi et al., "AppWorld: A controllable world of apps," 2024.
- [5] C.~Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.

- [6] J.~Yang et al., "SWE-agent: Agent-computer interfaces enable automated software engineering," 2024.
- [7] C.~Xia et al., "Agentless: Demystifying LLM-based software engineering agents," 2024.
- [8] Y.~Zhang et al., "AutoCodeRover: Autonomous program improvement," 2024.
- [9] E.~Smith et al., "Is the cure worse than the disease?" in *Proc.\ SOSp*, 2015.
- [10] S.~Ismayilzada et al., "Preservation violations in APR," 2023.
- [11] J.~Petke et al., "Patch overfitting in automated program repair," 2024.
- [12] Y.~Lou et al., "Automated program repair: A survey," 2024.
- [13] A.~Madaan et al., "Self-refine: Iterative refinement with self-feedback," 2023.
- [14] J.~Huang et al., "Large language models cannot self-correct reasoning yet," 2023.
- [15] N.~Shinn et al., "Reflexion: Language agents with verbal reinforcement learning," 2023.
- [16] H.~Pearce et al., "Asleep at the keyboard?" in *Proc.\ CCS*, 2022.
- [17] H.~Pearce et al., "Zero-shot vulnerability repair," 2023.
- [18] N.~Perry et al., "Do users write more insecure code with AI assistants?" 2023.
- [19] S.~A.~H.~Shah, "VibeGuardBench research protocol -- study in progress," github.com/iamcipherdev/VibeGuardBench, v1.0.2, Sept.\ 2026.